Politécnico do Porto
Instituto Superior de Engenharia do Porto

JavaFX

Eduardo Miguel Moreira Junqueira

Mestrado em Engenharia Electrotécnica e de Computadores
Área de Especialização em Automação e Sistemas



Departamento de Engenharia Eletrotécnica
Instituto Superior de Engenharia do Porto

Novembro, 2026

Esta dissertação satisfaz, parcialmente, os requisitos que constam da Ficha de Unidade Curricular de Tese/Dissertação, do 2º ano, do Mestrado em Engenharia Electrotécnica e de Computadores, Área de Especialização em Automação e Sistemas.

Candidato: Eduardo Miguel Moreira Junqueira, Nº 1251561,
1251561@isep.ipp.pt

Orientação Científica: Benedita Malheiro, mbm@isep.ipp.pt



Departamento de Engenharia Eletrotécnica
Instituto Superior de Engenharia do Porto
Rua Dr. António Bernardino de Almeida, 431, 4200–072 Porto

Novembro, 2026

Resumo

A *framework JavaFX*, será estudada e implementada neste relatório técnico-científico, com a criação de uma aplicação distribuída esta tecnologia será apresentada.

Aqui deverá ser apresentado o resumo de todo o trabalho efetuado. Esta secção não deverá exceder uma página.

Deve contextualizar o problema que pretende resolver ou a hipótese que irá formular, procure evidenciar as vantagens e desvantagens (se as houver) da solução encontrada, como também a forma através da qual a solução/hipótese foi validada. Neste último ponto, deverá referir-se aos desenvolvimentos efetuados, e à forma como validou (conformidade) e avaliou (desempenho) a solução encontrada.

A dissertação deve conter sempre duas versões do resumo: uma primeira no idioma do texto principal e a segunda num outro idioma. Este *template* assume que os dois idiomas em consideração são sempre Português e Inglês, assim, a classe irá colocar os cabeçalhos respetivos de acordo com o idioma selecionado nas opções da classe no ficheiro `main.tex`.

Palavras-Chave: *JavaFX*, Lista, separada por vírgulas, de palavras, frases, ou acrónimos chave no âmbito do trabalho descrito neste texto.

Abstract

The framework JavaFX, will be studied and implemented in this theoretical-cientific paper, with the creation of distributed aplication, this tecnology will be presented.

Keywords: JavaFX,

Índice

Lista de Figuras **vii**

Lista de Tabelas **ix**

Listagens **xi**

Lista de Acrónimos **xiii**

Lista de Símbolos **xv**

1 Introdução **1**
1.1 Contextualização 1
1.2 Objetivos 2
1.3 Planeamento 2
1.4 Organização do Documento 2

2 Problema **3**
2.1 Definição 3
2.2 Requisitos 3
2.2.1 Funcionais 4
2.2.2 Técnicos 4
2.3 Sumário 5

3 Solução **7**
3.1 Ferramentas e Tecnologias 7
3.2 Arquitetura 9
3.3 Modelo de Dados 10
3.4 Front-end 10
3.5 Back-end 13
3.6 Sumário 14

4 Testes e Avaliação **15**
4.1 Funcionais 15

5 Conclusões **17**

5.1 Sumário dos Resultados 17

5.2 Limitações 17

5.3 Desenvolvimento Futuro 17

Referências **19**

Lista de Figuras

3.1 Página de download do JavaFX SDK disponibilizado pela Gluon. . . 8
3.2 Erro de execução decorrente da falta de configuração do `-module-path`. 8
3.3 Janela de teste inicial ("Olá, JavaFX!") validando o ambiente gráfico. 9
3.4 Arquitetura distribuída de três camadas da aplicação *Gestão de Tarefas*. 9
3.5 Modelo relacional simplificado das entidades *Utilizador* e *Tarefa*. . . 10
3.6 Interface principal do sistema na perspetiva do perfil Administrador. 11
3.7 Ecrã de autenticação exibindo validação de campos obrigatórios. . . 11
3.8 Interface Web da aplicação em execução no Apache Tomcat (vista Administrador). 12
3.9 Interface Web no Apache Tomcat na perspetiva de um utilizador comum. 12
3.10 Configuração de privilégios de acesso e registos da tabela de utilizadores no MySQL Workbench. 13
3.11 Estrutura de pastas da aplicação evidenciando o módulo de suporte ao servidor Tomcat. 14

Lista de Tabelas

1.1 Cronograma das tarefas. 2

2.1 Casos de uso e correspondência com os métodos *Hypertext Transfer Protocol* (HTTP) da *Application Programming Interface* (API). . . . 4

3.1 *Endpoints* da API *Representational State Transfer* (REST) do *back-end*. 13

4.1 Resumo dos casos de teste funcionais. 15

Listagens

Lista de Acrónimos

API	*Application Programming Interface*
CRUD	*Create, Read, Update, Delete*
DAO	*Data Access Object*
HTTP	*Hypertext Transfer Protocol*
IDE	*Integrated Development Environment*
JDBC	*Java Database Connectivity*
JDK	*Java Development Kit*
JSON	*JavaScript Object Notation*
REST	*Representational State Transfer*
RINTE	*Redes Inteligentes e Serviços*
SGBD	Sistema de Gestão de Base de Dados
UC	*Unidade Curricular*

Lista de Símbolos

Símbolo	Descrição	Unidades
f	força	N
i	corrente	A
M	massa	kg
P	potência	W
x	deslocamento	m
θ	deslocamento angular	rad
ω	velocidade angular	rad/s

Capítulo 1

Introdução

A este capítulo introdutório a contextualização e organização do documento descrevem o relatório de uma forma geral, enquanto que os objetivos e o planeamento demonstram mais ao pormenor o estudo e a análise que foi feito ao longo deste relatório.

1.1 Contextualização

No âmbito da *Unidade Curricular* (UC) de *Redes Inteligentes e Serviços* (RINTE), será realizado o trabalho nº1, sobre uma tecnologia de suporte ao desenvolvimento de aplicações. A elaboração deste relatório técnico-científico é sobre a tecnologia *JavaFX Framework*. A entrega deste relatório será acompanhada também por código e apresentação exclusivamente para esta UC de RINTE da autoria própria do aluno, que realizou o trabalho, Eduardo Miguel Moreira Junqueira.
JavaFX, uma plataforma de software criada por Sun Microsystems em 2008. Em 2011 tornou-se *open-source* e no ano seguinte migrou para Oracle Corporation's *Java Development Kit* (JDK). Em 2018 Oracle Corporation's, decidiu não agregar mais esta tecnologia e então formou-se a comunidade representante atualmente com a parceria da GLUON, nomeada de *OpenJFX*[1],[2].

1.2 Objetivos

O objetivo principal deste trabalho prático é estudar e implementar a tecnologia *JavaFX Framework* de suporte ao desenvolvimento de aplicações distribuídas.
Esta aplicação distribuída, que será criada como exemplo demonstrativo da tecnologia em estudo, tem como âmbito seguir uma arquitetura de três camadas, ou seja, **I-** cliente gráfico em *JavaFX*, **II-** servidor em *Tomcat*, **III-** base de dados em *MySQL*.
Inicialmente será realizado um teste de implementação em conjunto com a pesquisa, para verificar a tecnologia a funcionar com as devidas bibliotecas necessárias.
De seguida, será implementado um projeto nomeado "Gestão de Tarefas", com dois tipos de clientes, o administrador e o utilizador.

1. Administrador: usa métodos HTTP do tipo *GET,POST,PUT,DELETE* para controlar todas as tarefas. Também usa métodos HTTP do tipo *GET,POST,DELETE* para controlar todos os utilizadores.

2. Utilizador: usa métodos HTTP do tipo *GET,POST,PUT* para controlar apenas as suas tarefas.

1.3 Planeamento

A tabela seguinte 1.1, demonstra as principais tarefas que foram realizadas de forma autónoma, ao longo de 5 semanas, após a atribuição da tecnologia em estudo.

Tarefas	28/9 a 4/10	5/10 a 11/10	12/10 a 18/10	19/10 a 25/10	26/10 a 1/11
Pesquisa	1	1	1	1	1
Relatório	0	0	0	1	1
Implementação	0	0	1	1	1
Validação	0	0	0	0	1

Tabela 1.1: Cronograma das tarefas.

1.4 Organização do Documento

Nos seguintes capítulos, o problema contém a definição e os requisitos quer funcionais, quer técnicos. A solução explicita as ferramentas e tecnologias utilizadas, identificando a arquitetura, os modelo de dados, *front-end, back-end*. Os testes e avaliação demonstram as funcionais o desempenho e também a usabilidade. A conclusão retrata o sumário dos resultados com as devidas limitações e o desenvolvimento futuro do trabalho nº2.

Capítulo 2

Problema

O projeto que será realizado de gestão de tarefas, pretende demonstrar a tecnologia numa arquitetura distribuída de três camadas. Neste capítulo é apresentada a definição do problema, os requisitos funcionais e técnicos da solução.

2.1 Definição

A gestão de tarefas, este projeto, no contexto adacémico é um exemplo criado com o intuido de explicar a tecnologia em estudo. O problema surge assim em dois campos distintos:

- Validação de solução gráfica da tecnologia em estudo.
- Integração do servidor *Tomcat* para comunicar com a base de dados *MySQL* para o serviço remoto da aplicação distribuída.

A solução proposta tem como intuito, validar os conceitos lecionados na UC de RINTE, nomeadamente a comunicação entre um cliente e um servidor através de um serviço *web* baseado no protocolo HTTP de três camadas.

2.2 Requisitos

Os requisitos da aplicação *Gestão de Tarefas* dividem-se em requisitos funcionais, que descrevem o comportamento esperado do sistema do ponto de vista dos seus

utilizadores, e requisitos técnicos, que definem as regras não funcionais que a solução deve respeitar.

2.2.1 Funcionais

A aplicação distribuída têm dois tipos de atores, com níveis de permissão distintos:

- **Administrador** – responsável pela gestão de todas as tarefas do sistema e pela gestão dos utilizadores registados;
- **Utilizador** – responsável apenas pela gestão das tarefas que lhe foram atribuídas ou que criou.

A Tabela 2.1 resume os principais casos de uso identificados, associando-os ao ator responsável e ao método HTTP que a interface REST do *back-end* segue neste projeto.

Tabela 2.1: Casos de uso e correspondência com os métodos HTTP da API.

Ator	**Caso de Uso**	**Método HTTP**
Administrador & Utilizador	Autenticar-se no sistema	`POST`
Administrador	Criar tarefa	`POST`
Administrador	Consultar todas as tarefas	`GET`
Administrador	Atualizar qualquer tarefa	`PUT`
Administrador	Eliminar qualquer tarefa	`DELETE`
Administrador	Criar utilizador	`POST`
Administrador	Consultar utilizadores	`GET`
Administrador	Eliminar utilizador	`DELETE`
Utilizador	Criar tarefa própria	`POST`
Utilizador	Consultar as suas tarefas	`GET`
Utilizador	Atualizar tarefa própria	`PUT`

2.2.2 Técnicos

Para além dos requisitos funcionais, a solução deve ainda respeitar os seguintes requisitos técnicos:

- **Autenticação e autorização** – o acesso à API deve ser condicionado à autenticação do utilizador, devendo o *back-end* validar, em cada pedido, se o ator autenticado possui permissões para a operação solicitada;
- **Portabilidade** – o cliente gráfico, por ser desenvolvido em *JavaFX*, deve poder ser executado em diferentes sistemas operativos sem alterações ao código;

Estes requisitos técnicos serão posteriormente retomados no Capítulo 4, onde são avaliados através de testes de desempenho e usabilidade.

2.3 Sumário

Neste capítulo definiu-se o problema, na necessidade de uma aplicação distribuída de gestão de tarefas que integra a tecnologia *JavaFX Framework*, e identifica-se os requisitos funcionais e técnicos que ilustram o desenho do problema.

Capítulo 3

Solução

Este capítulo apresenta a solução desenvolvida para o problema descrito no Capítulo 2, começa por enquadrar as ferramentas e tecnologias escolhidas, seguindo-se da descrição da arquitetura do sistema, do modelo de dados e, por último, da implementação das componentes de *front-end* e de *back-end*.

3.1 Ferramentas e Tecnologias

A escolha desta tecnologia é referenciada também pela literatura, que evidência vantagens relativamente a alternativas mais antigas, como o *AWT* e o *Swing*, nomeadamente ao nível da separação entre a definição visual, feita em *FXML*, e a lógica da aplicação [1]. A construção dos ecrãs recorre ao *Scene Builder*, uma ferramenta visual de edição de *FXML* documentada [3].
Do lado do servidor, é utilizado o *Apache Tomcat*, responsável por expor uma API REST que serve de interface entre o cliente gráfico e a base de dados. A comunicação entre as duas camadas é efetuada sobre HTTP, recorrendo ao formato *JavaScript Object Notation* (JSON) para a troca de mensagens, por ser um formato leve e nativamente suportado pelas bibliotecas *Java*.
A persistência dos dados em Sistema de Gestão de Base de Dados (SGBD) *MySQL*, acedida a partir do *back-end* através de *Java Database Connectivity* (JDBC), a interface padrão do *Java* para ligação a bases de dados relacionais. O desenvolvimento é realizado sobre o JDK, com recurso a um *Integrated Development Environment*

(IDE) para a edição e depuração do código, e ao *Git*, por forma a documentar a evolução do projeto ao longo das cinco semanas definidas no planeamento (Tabela 1.1).

Para justificar a origem das bibliotecas gráficas utilizadas e assegurar a reprodutibilidade do ambiente de desenvolvimento, foi documentado o processo de obtenção do respetivo SDK através do distribuidor oficial, conforme ilustra a Figura 3.1.

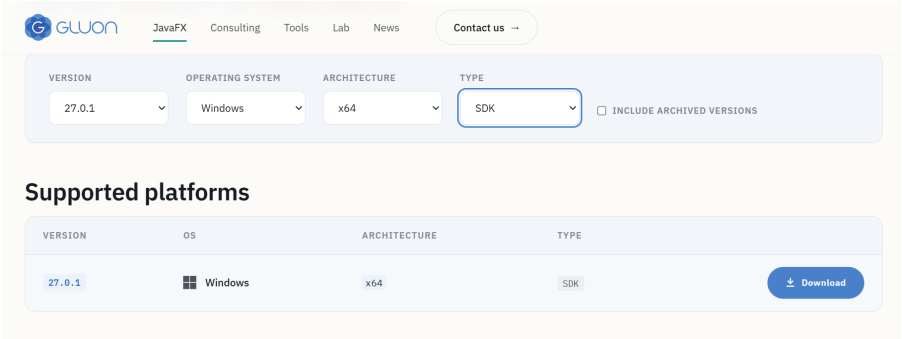


Figura 3.1: Página de download do JavaFX SDK disponibilizado pela Gluon.

Tal como referido anteriormente, a desassociação do JavaFX do JDK base a partir da versão 11 exige a configuração explícita de argumentos de execução e do caminho dos módulos. A ausência do parâmetro `-module-path` resulta no erro de compilação e execução evidenciado na Figura 3.2.

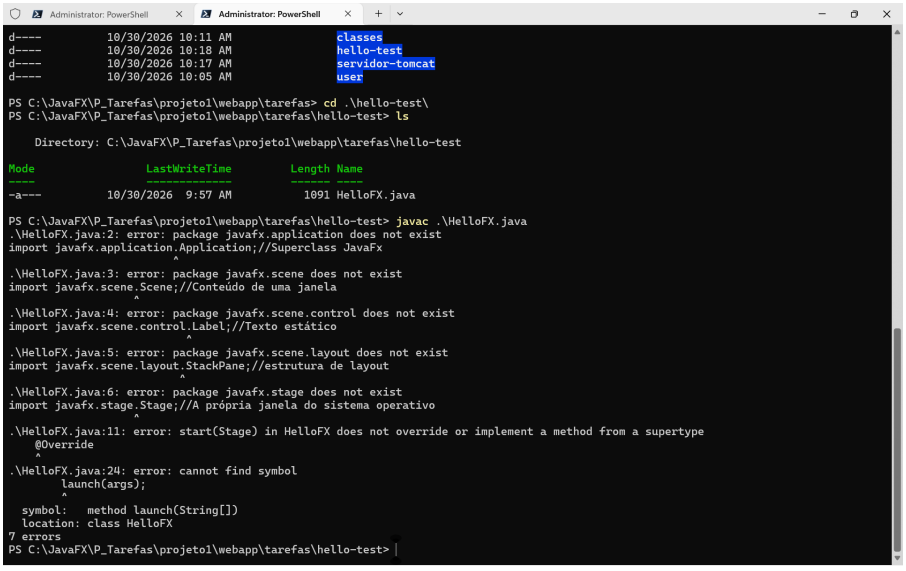


Figura 3.2: Erro de execução decorrente da falta de configuração do `-module-path`.

Após a correta parametrização das variáveis de ambiente e bibliotecas modulares, desenvolveu-se uma prova de conceito básica para validar o ciclo de vida da aplicação gráfica e a correta renderização na máquina de desenvolvimento (Figura 3.3).

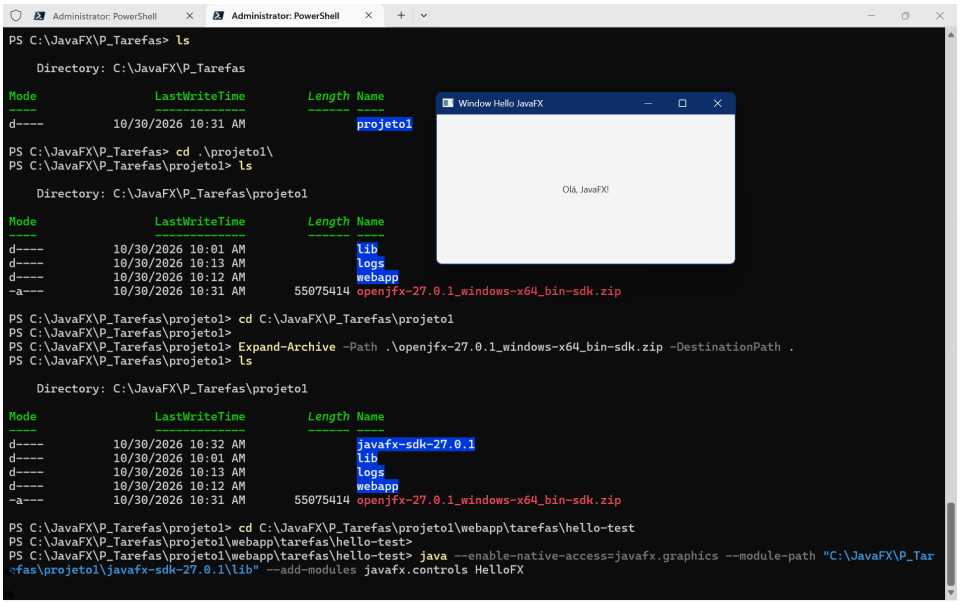


Figura 3.3: Janela de teste inicial ("Olá, JavaFX!") validando o ambiente gráfico.

3.2 Arquitetura

A aplicação segue uma arquitetura distribuída de três camadas, tal como definido nos objetivos do trabalho: apresentação, lógica de negócio e dados. A Figura 3.4 ilustra esta organização e o protocolo de comunicação utilizado entre cada camada.

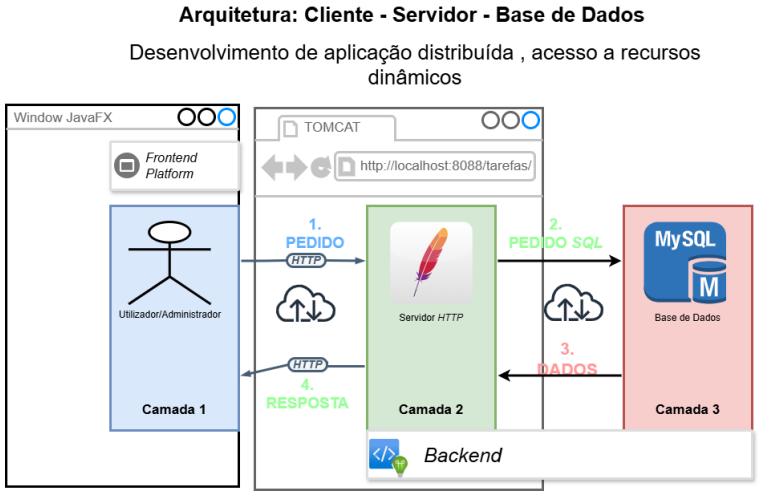


Figura 3.4: Arquitetura distribuída de três camadas da aplicação *Gestão de Tarefas*.

Como representado na Figura 3.4, o cliente *JavaFX* não acede diretamente à base de dados, comunicando exclusivamente com o servidor *Tomcat* através de pedidos HTTP. O servidor, por sua vez, está na camada intermédia, com acesso à camada de dados, que comunica com o SGBD *MySQL* através de JDBC.

3.3 Modelo de Dados

O modelo de dados da aplicação assenta em duas entidades principais: *Utilizador* e *Tarefa*, relacionadas entre si numa associação de um para muitos, já que um utilizador pode ser responsável por várias tarefas, mas cada tarefa pertence apenas a um utilizador. A Figura 3.5 apresenta o modelo relacional simplificado destas duas entidades.

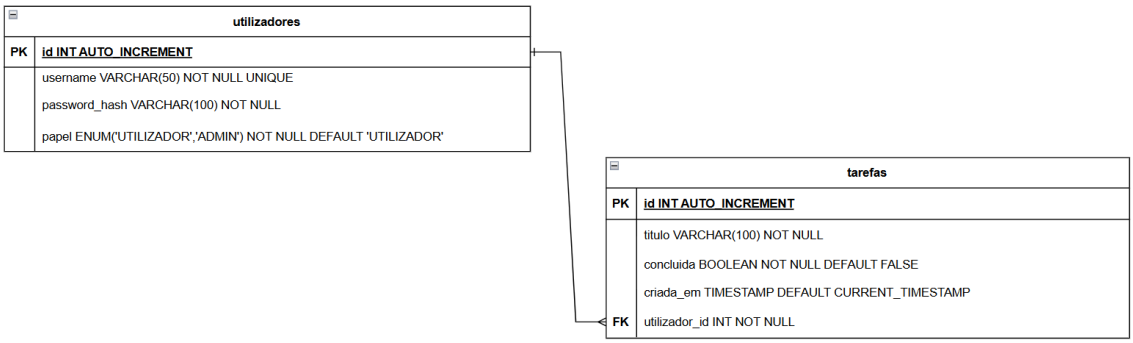


Figura 3.5: Modelo relacional simplificado das entidades *Utilizador* e *Tarefa*.

Conforme ilustrado na Figura 3.5, a entidade Utilizador armazena, entre outros campos, o atributo `perfil`, que distingue os registos do tipo Administrador dos registos do tipo *Utilizador* comum, sendo este o campo consultado pelo *back-end* para aplicar as regras de autorização descritas na Secção 3.2. A entidade Tarefa inclui a chave estrangeira `idUtilizador`, que a associa ao seu responsável, bem como o campo `estado`, utilizado para representar o progresso da tarefa (por exemplo, por fazer, a fazer ou concluída).

3.4 Front-end

O cliente gráfico tem a estrutura correspondendo a cada uma a um ecrã da aplicação, cada ecrã é associada a uma classe *controller JavaFX*, As respostas recebidas em JSON são convertidas para objetos *Java* antes de serem apresentadas.

- **Autenticação:** ecrã inicial onde o utilizador introduz as suas credenciais, submetidas ao *endpoint* de autenticação do *back-end*;
- **Lista de Tarefas:** apresenta, numa `TableView`, as tarefas do Utilizador autenticado ou, no caso do Administrador, a totalidade das tarefas registadas no sistema;

- **Formulário de Tarefa:** permite a criação e a edição de uma tarefa, sendo reutilizado tanto para o Administrador como para o Utilizador;
- **Gestão de Utilizadores:** ecrã exclusivo do Administrador, que lista os utilizadores registados e permite a sua criação e remoção.

A interface gráfica foi desenhada para adaptar as funcionalidades disponíveis ao perfil de acesso autenticado. Na Figura 3.6 apresenta-se a vista do utilizador com perfil de Administrador, destacando o acesso concorrente à gestão de tarefas e ao controlo de contas de utilizadores.

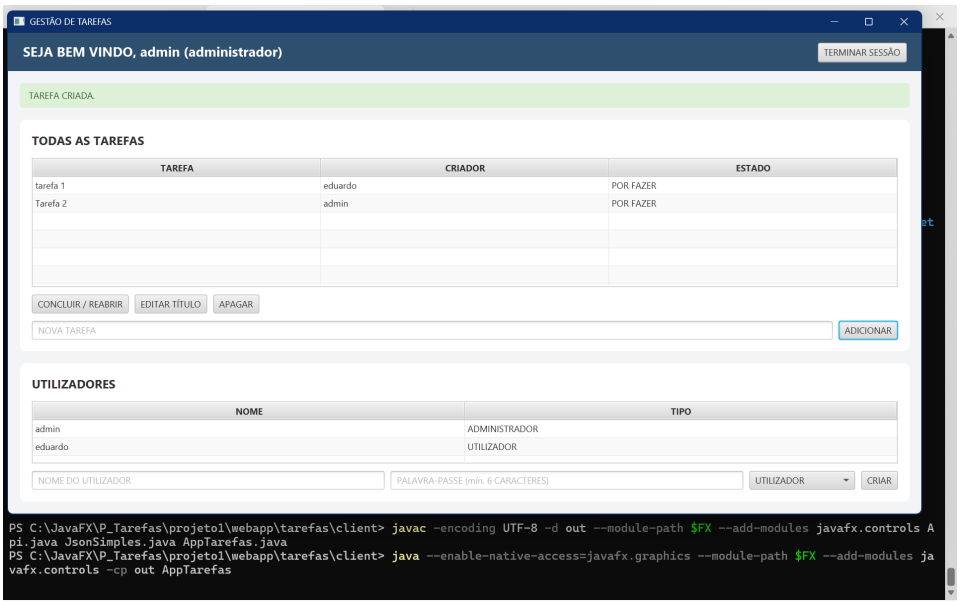


Figura 3.6: Interface principal do sistema na perspetiva do perfil Administrador.

Para além do desenho estético, a interface de autenticação contempla o tratamento imediato de erros de entrada de dados, alertando o utilizador em tempo real caso tente submeter o formulário sem preencher os campos obrigatórios (Figura 3.7).



Figura 3.7: Ecrã de autenticação exibindo validação de campos obrigatórios.

A aplicação foi disponibilizada através de um servidor de aplicações Apache Tomcat, permitindo o acesso via navegador web. Na Figura 3.8 é apresentada a interface Web acedida por uma conta de administrador, evidenciando as funcionalidades de gestão centralizada de todas as tarefas da plataforma, bem como o módulo de gestão e criação de novos utilizadores com diferentes níveis de privilégios.

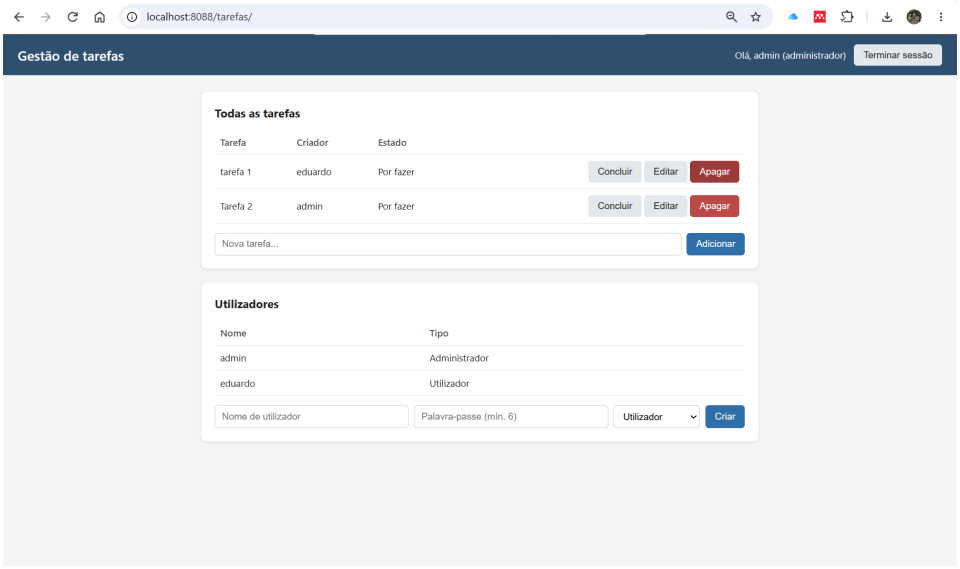


Figura 3.8: Interface Web da aplicação em execução no Apache Tomcat (vista Administrador).

Em conformidade com as permissões atribuídas a cada conta, a interface Web adapta os elementos visíveis de acordo com o papel do utilizador autenticado. A Figura 3.9 ilustra a perspetiva de um utilizador comum (`eduardo`), demonstrando a restrição de acesso: este visualiza apenas as tarefas sob a sua responsabilidade e não dispõe de privilégios para apagar tarefas nem para aceder ao módulo de gestão de utilizadores.

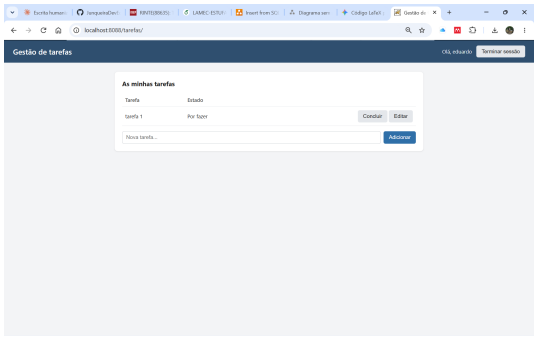


Figura 3.9: Interface Web no Apache Tomcat na perspetiva de um utilizador comum.

3.5 Back-end

O *back-end* é implementado como um conjunto de *servlets* integrados dentro do *Apache Tomcat*, organizados segundo o padrão *Data Access Object* (DAO) para o acesso aos dados, isolando as operações *Create, Read, Update, Delete* (CRUD) sobre as entidades Utilizador e Tarefa da restante lógica de *front-end*. A Tabela 3.1 resume os principais *endpoints* disponibilizados pela API REST.

Tabela 3.1: *Endpoints* da API REST do *back-end*.

Método	Recurso	Acesso
`POST`	`/auth/login`	Administrador & Utilizador
`GET`	`/tarefas`	Administrador (todas) / Utilizador (próprias)
`POST`	`/tarefas`	Administrador & Utilizador
`PUT`	`/tarefas/{id}`	Administrador (todas) / Utilizador (próprias)
`DELETE`	`/tarefas/{id}`	Administrador
`GET`	`/utilizadores`	Administrador
`POST`	`/utilizadores`	Administrador
`DELETE`	`/utilizadores/{id}`	Administrador

No âmbito da segurança e do princípio do menor privilégio, a ligação da base de dados é efetuada por uma conta de serviço dedicada, com acessos estritamente delimitados às tabelas necessárias (Figura 3.10).

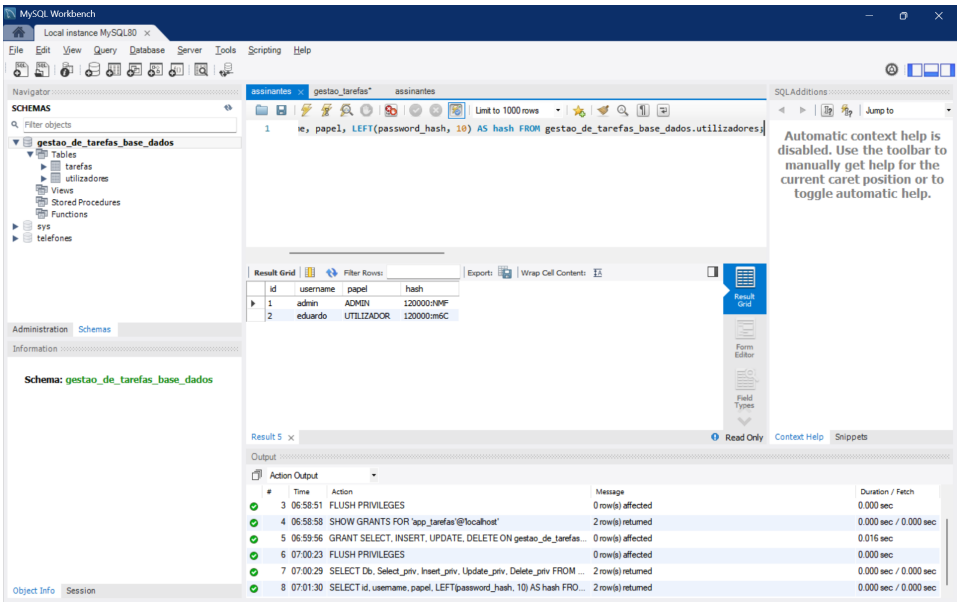


Figura 3.10: Configuração de privilégios de acesso e registos da tabela de utilizadores no MySQL Workbench.

De forma a assegurar a modularidade e a clara separação de responsabilidades na camada de suporte ao servidor, o diretório de implementação organiza-se numa

estrutura hierárquica bem delineada. Na Figura 3.11 apresenta-se a disposição dos diretórios principais.

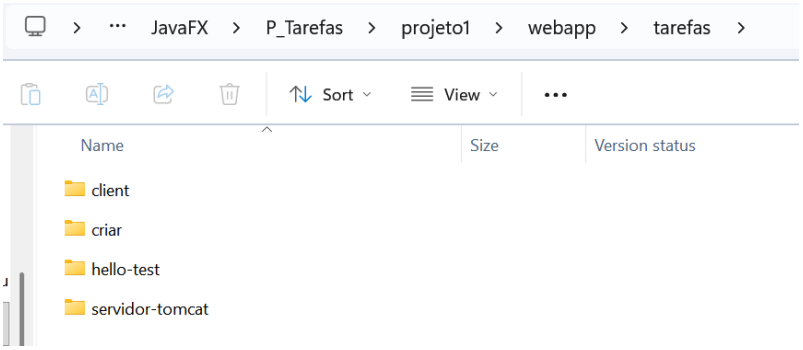


Figura 3.11: Estrutura de pastas da aplicação evidenciando o módulo de suporte ao servidor Tomcat.

3.6 Sumário

Neste capítulo descreveu-se a solução implementada para a aplicação *Gestão de Tarefas*, assente numa arquitetura distribuída de três camadas que recorre ao *JavaFX* na apresentação, ao *Tomcat* na lógica de servidor e ao *MySQL* na persistência de base de dados. As camadas assim comunicam entre si por HTTP/JSON e JDBC, respetivamente.

Capítulo 4

Testes e Avaliação

Este capítulo descreve o processo de validação da aplicação *Gestão de Tarefas*, estruturado em três vertentes complementares: testes funcionais, avaliação de desempenho e avaliação de usabilidade.

4.1 Funcionais

A validação funcional teve por base os casos de uso identificados na Tabela 2.1. A Tabela 4.1 resume os principais casos de teste executados.

Tabela 4.1: Resumo dos casos de teste funcionais.

Ator	**Cenário**	**Resultado**
Utilizador	Autenticação com credenciais válidas	[a preencher]
Utilizador	Criação de uma tarefa própria	[a preencher]
Utilizador	Consulta das tarefas próprias	[a preencher]
Utilizador	Tentativa de `DELETE` sobre uma tarefa	[a preencher]
Utilizador	Tentativa de acesso a tarefas de outro utilizador	[a preencher]
Administrador	Consulta de todas as tarefas do sistema	[a preencher]
Administrador	Criação e remoção de um utilizador	[a preencher]
Administrador	Eliminação de uma tarefa de outro utilizador	[a preencher]

Capítulo 5

Conclusões

Este capítulo encerra o relatório técnico-científico, identificando as limitações do trabalho desenvolvido e o desenvolvimento futuro a realizar no trabalho nº2 da UC de RINTE.

5.1 Sumário dos Resultados

O trabalho desenvolvido cumpriu o objetivo principal definido no Capítulo 1: estudar e implementar a tecnologia *JavaFX Framework* de suporte ao desenvolvimento de aplicações distribuídas, através da aplicação *Gestão de Tarefas*. Esta aplicação demonstrou, numa arquitetura de três camadas, a integração de um cliente gráfico *JavaFX* com um *back-end Tomcat* e uma base de dados *MySQL*, comunicando por HTTP/JSON e JDBC, respetivamente, conforme descrito no Capítulo 3.

5.2 Limitações

O curto período de desenvolvimento de cinco semanas, conforme o cronograma da Tabela 1.1, foi o principal que impediu de melhorar este trabalho e o seu conteúdo.

5.3 Desenvolvimento Futuro

Como desenvolvimento futuro, a explorar no âmbito do trabalho nº2 da UC de RINTE, identificam-se as seguintes linhas de evolução:

- aprofundar a segurança da comunicação entre o cliente e o *back-end*, nomeadamente através de *tokens* de autenticação com tempo de vida limitado, em complemento ao mecanismo de autenticação atual;
- estender o modelo de dados apresentado na Secção 3.3, de forma a suportar categorias de tarefas, prioridades e histórico de alterações;

Referências

[1] A. L. Pereira, "2d animation of recursive backtracking maze solution using javafx versus awt and swing," pp. 1787–1793, Institute of Electrical and Electronics Engineers Inc., 12 2020. [Citado nas páginas 1 e 7]

[2] C. OpenJFX, "Javafx." Disponível em `https://openjfx.io/`. (Último Acesso em 1/11/2026). [Citado na página 1]

[3] V. Vivien, *JavaFX 1.2 application development cookbook : over 80 recipes to create rich internet applications with many exciting features.* 2010. [Citado na página 7]